SAMCO — Current UI Functional Acceptance & Browser Automation Test Report

Project	SAMCO (restaurant queue, banquet, tasks and reporting platform)
Test scope	The currently implemented SAMCO web application, API and database
Test objective	Verify that the functionality already built into SAMCO works correctly end to end, through the real user interface
Test environment	Next.js web application, NestJS API, PostgreSQL, Chromium via Playwright
Testing period	11–12 September 2026
Overall status	**PASS** for the defined acceptance scope
Report prepared for	Senior / technical lead review

1. Executive summary

SAMCO had accumulated several implemented modules (queue, banquet, tasks and routines, reports and dashboard) without a structured acceptance pass over the user interface itself. Automated tests existed for the API and for frontend components in isolation, but nothing exercised the assembled product the way a member of restaurant staff does. This work closed that gap.

Testing was carried out against the **actual running application**: the real web application, the real API and a real PostgreSQL database, with data created through the application's own endpoints rather than injected into the database. Two phases were performed:

1. **An exploratory acceptance pass** across every currently exposed screen and control.
2. **New browser automation**, introduced with **Playwright driving real Chromium**, to cover the two areas that could not be completed by hand and to make the most security-sensitive behaviour repeatable.

Outcome. All 27 automated browser acceptance tests pass, and every existing automated suite remains green (618 API unit tests, 596 API end-to-end tests, 321 web tests). Nine genuine application defects were found across the two phases; all nine were fixed, each with regression coverage, and re-verified in the browser.

Major defects discovered were: six navigation destinations that led to a dead-end error page; a branch selector offering branches the user had no access to; a mobile layout defect that forced the whole application off-screen on a phone; a scheduling defect that fabricated overdue work; refusal screens that also claimed "nothing exists"; and a navigation defect that sent kitchen staff to a page they were not allowed to open. Details are in section 6.

Stability. The functionality inside the defined scope is behaving consistently: the browser suite completed four consecutive clean runs after the fixes, with retries disabled.

Scope caution. This report certifies the **currently implemented** functionality against the acceptance scope defined below. It is not a statement that the SAMCO product as a whole is production-ready, and it does not cover unimplemented or planned functionality.

2. Test objective

The objective was to verify that the functionality **already implemented** in SAMCO works correctly from end to end — user interface through to stored data — and that its access rules hold.

The acceptance covered:

- user interface interaction and frontend behaviour;
- API integration behind each screen;
- database persistence, verified directly for the principal flows;
- authentication and session behaviour;
- authorisation and permission-driven interface behaviour;
- company and branch isolation;
- realtime (server-sent event) behaviour;
- responsive behaviour across desktop, tablet and narrow viewports;
- display-device security;
- role-based interface behaviour.

> **Functionality not currently implemented in SAMCO was not introduced or tested merely because it appears in the SRS.** Where a screen does not exist, that was recorded as a finding rather than filled in. Implementation work belonging to a separate, third-party codebase was explicitly out of scope for this exercise.

3. Test scope

The areas below were exercised. Items are listed only where they were actually tested.

3.1 Authentication and access

- Sign-in with valid credentials; redirect to the branch home screen.
- Rejected sign-ins: empty fields, unknown user, wrong password, and a deactivated account — all returning one generic message, with no indication of which accounts exist.
- Sign-in attempted while the API was unreachable (honest failure message).
- Session persistence across page reloads, and silent token refresh mid-action.
- Sign-out, including server-side session revocation confirmed in the database.
- Permission-driven access to each module.
- Company and branch context selection.

3.2 Foundation and application shell

- Navigation contents per role, and the workspace launcher.
- Company and branch switching, with data following the selected branch.
- Customer records — created and selected through the queue registration and banquet booking flows.
- Tables — exercised through podium table assignment and release.
- Display devices — provisioned through the API; note that no management screen for customers, tables or devices exists in the current build (see section 6, NAV-01).
- Shell behaviour: loading, error and empty states.

3.3 Queue

- Customer registration (walk-in, existing customer, new customer), party size controls and validation, advance-booking guard.
- Token creation and numbering, verified in the database.
- Podium: call, seat with and without a table, table assignment and release, no-show, cancellation.
- Invalid state transitions and occupied or inactive tables, all refused by the API.
- Queue board and podium views, including empty states.
- Customer and administrative display screens.
- Realtime updates to a second session and to paired screens.

3.4 Banquet

- Halls and packages: creation, validation, duplicate names, deactivation and retirement.
- Bookings: creation, guest-count and time validation, hall-conflict refusal on confirmation.
- Status transitions: draft, confirmed, completed, cancelled.
- Payments: validation, over-payment refusal, currency mismatch refusal, and running balance.
- Event plans, including kitchen notes.
- Kitchen / KDS: starting and readying dishes, with live counts.
- Chef display screen and its realtime updates.

3.5 Tasks and routines

- Task creation, priority and due dates; checklist templates with required and optional items.
- Task lifecycle: start, complete, cancel.
- Required-checklist validation blocking completion.
- Assignment clearing (note: no screen currently creates an assignment — see section 12).
- Routines: creation (daily and weekly), pause and resume, and generation of due work.
- Completion summary reporting.
- Realtime board updates.

3.6 Reports and dashboard

- Dashboard widgets, cross-checked against database contents.
- All seven report definitions available to a full-permission user.

- Report filters, date ranges, invalid ranges and empty periods.
- Pagination across more than one page of results.
- Excel and PDF export (file type and content signature verified; no file was retained).
- Permission filtering of the report catalogue.

3.7 Responsive interface

- Desktop, tablet (768 px) and narrow mobile (375 px) viewports across the staff screens and the display screens.

---

4. Testing approach

A layered strategy was used, and the layers are reported separately because they carry different weight as evidence.

4.1 Existing automated tests (pre-existing, re-run as regression)

- **API unit tests** — service and domain logic in isolation.
- **API end-to-end tests** — the whole API booted in-process against a real database, covering authorisation, queue, banquet, tasks, reports, realtime and display-device rules.
- **Web component tests (Vitest)** — frontend components and hooks with the API mocked.

4.2 Exploratory (manual) acceptance

A structured, human-directed pass over every exposed route and control, driven through a Chromium browser pane with direct inspection of network traffic, database rows and audit records. **148 checks** were recorded: 74 passed by direct interaction, 57 passed where controls were activated programmatically through the application's own event handlers (a limitation of that browser pane, recorded transparently at the time), 9 failed and became defects, 4 were recorded as observations, and 2 areas were blocked. This phase is **exploratory evidence, not repeatable automation**, and its value is in discovery.

4.3 New browser automation (Playwright)

Playwright was introduced to drive **real Chromium** against a **production build** of the web application and the real API, both started by the test run itself on dedicated ports. The suite covers the two previously blocked areas and the security behaviour that matters most.

The browser tests use:

- **isolated browser contexts** — every screen and every staff role gets its own context, so no session or paired device is shared;
- a **dedicated end-to-end database**, created, migrated and reset by the suite;
- **generated test credentials** — a fresh password per run, held only in the test processes' environment;
- the **real login flow** — the actual form, typed and submitted, with no injected session state;
- the **real display pairing flow** — the actual pairing form, using credentials minted through the application's own device-provisioning endpoint;
- **real API interaction** for the staff-side changes a screen must reflect;

- **real user-interface interaction** for every assertion;
- **realtime (SSE) verification** using bounded, event-driven waits rather than fixed delays;
- **responsive viewport testing**, including phone emulation.

> **The browser suite never uses the development database.** It runs against a dedicated end-to-end database, and every destructive step is protected by a guard that refuses any database whose name does not identify it as an end-to-end database. The guard is checked twice: against the configured connection, and against the database the live connection actually reached.

---

5. Browser automation results

Test area	Tests	Passed	Failed
Displays	15	15	0
Chef	4	4	0
Worker	5	5	0
No-permission	3	3	0
TOTAL	**27**	**27**	**0**

5.1 Display screens (15 tests)

- **Pairing** — each of the three screen types paired through the real pairing form.
- **Correct projection** — the customer screen shows the public "now serving" board; the administrative screen shows the operational board; the kitchen screen shows the evening's dishes and covers.
- **Privacy** — no customer names, telephone numbers or party sizes on the customer-facing screen; no customer identity and no monetary information on the kitchen screen. The live event payloads were confirmed to carry no customer identity either.
- **Branch isolation** — a screen shows only its own branch's data, verified against a second branch holding a different number of waiting parties.
- **Reload persistence** — a paired screen returns to its board after a full page reload.
- **Realtime updates** — parties called and seated by staff appear on the screens without any reload, within a bound well below the screens' fallback polling interval, so the result proves the push path.
- **Outage and recovery** — the API was genuinely stopped mid-test: the screen reports that it is reconnecting, keeps its last board, and does not mistake an outage for a rejected credential; when the API returns, the screen recovers by itself and resumes live updates.
- **Wrong-kind credential** — a genuine credential used on the wrong type of screen is refused.
- **Wrong branch** — a valid credential used against another branch of the same company is refused.
- **Other company** — the same credential against another company's branch is refused.
- **Empty and malformed credentials** — refused by the screen before any request reaches the API.
- **Revocation** — a revoked credential cannot pair; and a screen that is already open and live stops showing its board once its device is revoked.

- **Staff-shell separation** — a paired display cannot reach any staff route; every attempt lands on the sign-in page, and no staff session exists in the display's browser context.
- **Responsive behaviour** — the queue and kitchen screens fit a phone-sized viewport without sideways scrolling.

No credential, token or password appears in this report, in the test sources, or in any retained test artifact; the suite fails itself if a generated secret is found in an artifact.

5.2 Role-based browser workflows (12 tests)

- **Chef (4)** — navigation limited to Home and Banquet; Banquet opens the Kitchen board; starting and readying a dish updates the board and its counts; every other area (queue, tasks, reports, banquet catalogue) is refused with no data and no misleading empty state; the kitchen remains usable on a phone.
- **Worker (5)** — navigation limited to Home and Tasks; task sections limited to those permitted; no task-creation or cancellation controls; a task started, its required checklist items ticked, and the task completed, with the final state confirmed through the API; routines and checklists readable but not manageable; other areas refused; usable on a phone.
- **No-permission account (3)** — only Home is offered; no dashboard figures or workspace cards; seven protected routes refused with no data, no false empty state and no action controls; usable on a phone.

---

6. Defects found and resolved

Nine genuine application defects were identified across the two phases. All were fixed, each with regression coverage, and re-verified through the browser.

ID	Severity	Area	Defect	Status
NAV-01	P1	Navigation	Six navigation destinations had no page; selecting them produced a bare framework 404 outside the application shell, with no way back	Fixed
CHEF-NAV-01	P2	Banquet navigation	Kitchen-only staff were taken to a refused bookings page	Fixed
CTX-03	P2	Branch selector	Branches where the user holds no role were offered for selection	Fixed
RESP-04	P2	Application shell	Narrow-viewport layout overflow caused by the bottom navigation's minimum width	Fixed
ROUTINE-01	P2	Routines	Generating due work fabricated overdue tasks for days before a routine existed	Fixed
NOACCESS-01	P3	Multiple screens	Refused pages also displayed their empty state, claiming nothing existed	Fixed
COPY-01	P3	Queue registration	Confirmation text did not agree in number ("1 kids")	Fixed
AUDIT-01	P3	Audit log	Queue token numbers were redacted as if they were secrets	Fixed

ID	Severity	Area	Defect	Status
BQ-NAME-01	P3	Banquet API	Hall and package names consisting only of spaces were accepted	Fixed

CHEF-NAV-01 — the defect found by the new browser automation

Severity: P2

Problem. A chef clicking **Banquet** in the main navigation was taken to the bookings page, which they are not permitted to view, and which therefore displayed **Forbidden**. The Kitchen board — the one banquet section a chef may use — was reachable only by noticing a sub-tab. For a kitchen-only role, the single navigation entry available led to a refusal.

Root cause. The main navigation entry and the home workspace card both target the banquet bookings route, which requires booking-view permission. The banquet area filtered its sub-navigation by permission but never moved a permission-limited user to a section they were allowed to open.

Resolution. Navigation was corrected so that a user who may use the banquet area but not its bookings is taken to the first section they are permitted to open — the Kitchen board. This is a navigation change only: **no permission was widened, and the API continues to refuse bookings to that role.**

Regression coverage added for four cases: a kitchen-only user is redirected; a user with booking access is unaffected; no redirect occurs from other banquet routes; and no redirect occurs for a user with no banquet permission at all (who correctly still sees the refusal).

Browser retest: passed in four consecutive runs.

Selected detail on the earlier defects

- **NAV-01 (P1).** Six navigation destinations pointed at pages that had never been created. Each now opens an honest "not available in this version" page inside the application shell, with a route back. A regression test asserts that every navigation entry resolves to a real route.
- **RESP-04 (P2).** On a 375 px phone, a manager's eleven-item bottom navigation forced a 680 px layout, zooming the entire application out. The bar now scrolls within itself; the page fits the screen.
- **ROUTINE-01 (P2).** A routine created today generated "overdue" tasks for the three preceding days, inflating overdue counts and routine-adherence reporting. Catch-up no longer reaches back beyond the day a routine was created; the existing behaviour for genuinely missed days is preserved.
- **NOACCESS-01 (P3).** Screens reached without permission displayed the refusal **and** their empty state, telling the reader that nothing existed when in fact they simply could not see it. An empty state is now shown only after data has actually loaded.

---

7. Other test observations (not application defects)

The first Playwright run surfaced two issues in the **test code**, which are reported separately so they are not mistaken for product defects:

1. **Alert locator ambiguity.** The assertion for a screen's pairing error also matched the framework's always-present, empty route announcer, which carries the same accessibility role. The assertion was

scoped to the pairing form.

2. **Checklist checkbox timing.** The checklist checkbox reflects server state and is ticked only once the API confirms the change. The test used an immediate check-and-verify action, which inspected the control before confirmation arrived. It now clicks and waits for the confirmed state.

Both were corrected in the test suite. **Neither indicates a defect in the application**; in the second case the underlying behaviour (confirm-then-display) is the intended design.

---

8. Stability

Run	Result	Note
1	17 / 27	Before the application fix and the test-code corrections; this run is what exposed CHEF-NAV-01
2	27 / 27	After the fix and corrections
3	27 / 27	
4	27 / 27	
5	27 / 27	After a final change to the test harness

After the corrections, **four consecutive clean runs** completed with:

- **retries disabled** (a test that passes only on retry would be reported as a failure);
- **no browser test failures** and no flaky results;
- a **clean secret scan** of the run's artifacts on every run.

This demonstrates consistency across repeated executions on one machine over a single session. It is **not** evidence of long-term reliability across environments, machines or extended periods; that requires the suite to run repeatedly over time, ideally in continuous integration.

---

9. Full automated regression results

Test / check	Result
API unit tests	**618 / 618** passed (29 files)
API end-to-end tests	**596 / 596** passed (11 files)
Web tests (Vitest)	**321 / 321** passed (26 files)
API lint	Clean
Web lint	Clean (including the new browser tests)
Web typecheck	Clean (including the new browser tests)
API source typecheck	Clean
API production build	Pass

Test / check	Result
Web production build	Pass
Prisma schema validation	Pass
Prisma migrations	**12 / 12** applied
Database drift check	None detected
Browser acceptance (Playwright)	**27 / 27** passed

9.1 Pre-existing type errors in API test files

The API typecheck that **includes test files** reports **13 errors across 6 spec files**. For clarity:

- these errors **predate this browser automation work** — all six files are unchanged from the baseline commit at the start of this work;
- the **API production source typecheck is clean**, and the production build succeeds;
- they **did not prevent the API end-to-end suite from passing** (596 / 596), because the test runner does not typecheck;
- they remain **technical debt** and should be addressed as a separate, small piece of work.

Correction of an earlier figure: an interim note during the session recorded "5" such errors. That number came from a truncated view of the same output, not from a smaller set. **13 is the verified figure.**

9.2 One transient web test failure

During the regression phase, one web component test failed **once** while the web suite was deliberately run concurrently with the API end-to-end suite on the same machine. The same test passed when run on its own, and the full web suite subsequently passed **321 / 321** on an unloaded machine. This is recorded as **test-execution contention**, not a product defect; no code or test was changed in response.

---

10. Security and isolation validation

The following was verified during acceptance:

- **Authentication through the real interface** — no injected or mocked session state in any browser test.
- **Role-based access** — navigation, sub-navigation and action controls reflect the holder's permissions for three restricted roles.
- **Unauthorised routes** — protected routes reached directly by URL are refused, present no protected data, offer no action controls, and do not claim that data is absent.
- **Display credential validation** — malformed and empty entries are rejected before any request leaves the browser.
- **Wrong display type** — a genuine credential used on an incompatible screen is refused.
- **Wrong branch** — a credential is valid only for the branch it was paired to.
- **Other company** — the same credential is refused against another company's branch.

- **Revoked devices** — revoked credentials cannot pair, and an already-open screen stops serving its board after revocation.
- **Staff / display separation** — a paired display cannot reach the staff application, and holds no staff session.
- **Customer-display privacy** — no customer identity or party composition on the public screen; the kitchen screen carries no customer identity and no monetary values. Live event payloads were inspected and carry no customer identity.
- **Company and branch isolation** — verified both through the interface and directly against the API, including deliberately forged context headers, which were refused without leaking data.
- **Dedicated end-to-end database** — the browser suite provisions and resets its own database.
- **No destructive testing against development or production data** — a guard refuses any database that is not identified as an end-to-end database, checked both before connecting and against the live connection.

No identifiers, credentials, tokens or connection details are reproduced in this report.

---

11. Responsive validation

Staff screens and display screens were checked at desktop, tablet (768 px) and narrow mobile (375 px) viewports. Checks covered horizontal overflow, reachability of controls, minimum touch-target sizes, and that wide content (such as report tables) scrolls within its own container rather than moving the page.

RESP-04 — narrow mobile layout overflow. On a 375 px viewport, the bottom navigation's minimum item width forced the layout to roughly 680 px for roles with many navigation entries, so the entire application was scaled down or scrolled sideways. The navigation was corrected to scroll within its own bar, leaving the page at the width of the screen, with every navigation entry still reachable. Verified in the browser at phone size, and guarded by a regression test.

---

12. Known remaining items

These are **documented, non-blocking** items. None prevented the browser acceptance suite from completing, and none is a test failure.

Item	Type	Note
Mistyped URLs can still produce the framework's bare 404 outside the application shell	UI	Correcting this requires an experimental framework feature, which was deliberately not enabled
Permission refusals still use the raw "Forbidden" wording	UI copy	The wording is asserted by existing tests; changing it is a small, separate piece of work
`/tasks` opened without permission can remain on "Reconnecting…"	UI	The live-updates client keeps retrying a refused subscription

Item	Type	Note
No screen currently assigns a task, although the API supports assignment	Functional gap	Not implemented in the current build; deliberately not added during acceptance
Logo artwork is not supplied, so the text wordmark fallback is displayed	Asset	The fallback behaviour is intentional and tested
13 pre-existing type errors in API test files	Technical debt	See section 9.1; production source typecheck is clean

No new feature work is recommended by this report.

13. Test environment

Component	Detail
Web application	Next.js application, exercised as a production build during browser testing
API	NestJS service, compiled and run for the test session against the test database
Database	PostgreSQL
Browser	Chromium, installed and managed by Playwright
Browser automation	Playwright test runner, one worker, retries disabled
Test database	A dedicated end-to-end database, created, migrated and reset by the suite; separate from development data
Test isolation	One browser context per screen and per staff role; no shared session or paired-device state
Test data	Companies, branches, roles, accounts, tables, customers, queue, banquet and task data built fresh for each run through the application's own API
Secrets handling	Credentials generated per run and held only in the test processes' environment; traces and video disabled; artifacts scanned for leaked secrets at the end of every run

Diagnostic traces and video recording are disabled deliberately, because a trace would capture request headers and typed values.

14. Conclusion

The currently implemented SAMCO user interface has successfully completed the defined browser functional acceptance scope.

- **27 of 27 automated browser acceptance tests pass**, including display-device security and the role-based browser workflows for kitchen, task-worker and no-permission users.
- **The existing automated regression suites remain green**: 618 / 618 API unit tests, 596 / 596 API end-to-end tests and 321 / 321 web tests, with clean lint, clean production typechecks, successful builds,

and clean Prisma schema, migration and drift checks.

- **One genuine application defect (CHEF-NAV-01, P2) was identified and corrected during browser acceptance**, with regression coverage and browser re-verification. Eight further defects found in the preceding exploratory pass were also corrected and re-verified.
- **Several pre-existing, non-blocking technical and user-interface issues remain**, documented in section 12 with the technical debt in section 9.1.

This conclusion is limited to the functionality currently implemented in SAMCO and to the acceptance scope defined in section 3. It does not assess unimplemented functionality, non-functional characteristics such as performance, load and accessibility conformance, or deployment readiness.

Recommended next steps

1. Run the browser acceptance suite in continuous integration, so its stability is demonstrated over time rather than within one session.
2. Clear the 13 pre-existing type errors in the API test files.
3. Address the documented user-interface items in section 12 as a small, separate batch.

---

15. Management summary

Overall result

PASS — Current UI Functional Acceptance

Key numbers

Measure	Result
Browser acceptance tests	27 / 27 passed
API unit tests	618 / 618 passed
API end-to-end tests	596 / 596 passed
Web tests	321 / 321 passed
Lint (API and web)	Clean
Production builds (API and web)	Passed
Prisma migration and drift checks	Passed
Repeat browser runs after fixes	4 consecutive clean runs

Major finding

One **P2 navigation defect affecting kitchen (Chef) users** was identified by the new browser automation and fixed, with regression coverage. Kitchen staff previously landed on a refusal page when opening the only area available to them.

Eight further defects — one high-priority (six dead-end navigation destinations), three medium and four low — were found in the preceding exploratory pass and were also fixed and re-verified.

Remaining technical debt (non-blocking)

- 13 pre-existing type errors in API test files.
- Mistyped URLs can still show a bare framework error page outside the application shell.
- Permission refusals use raw "Forbidden" wording.
- Tasks screen can remain on "Reconnecting…" when opened without permission.
- No screen assigns a task, although the API supports it.
- Logo artwork not supplied; text fallback in use.

What this result does and does not say

It confirms that the **currently implemented** SAMCO interface behaves correctly across the tested scope, including its access and isolation rules. It is **not** a statement that the whole product is production-ready, that all planned requirements are complete, or that no defects remain outside the tested scope.